



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: __1__

Team No.: __20__

Project Title: Kernel Monitor: A Lightweight Linux Kernel Module for Real-Time Process and Resource Monitoring

Team Members

Roll Number	Student Name	Signature
2520030249	Namith	
2520030391	Subhash	
2520030539	Yeshwanth	

1. Abstract

(Summarize the project idea, problem background, relevant Operating Systems/System Programming concepts, proposed Linux-based solution, and expected outcome. The abstract should not exceed 250 words.)

Modern operating systems run many concurrent processes, and administrators typically rely on user-space utilities such as top and ps that repeatedly poll the /proc filesystem to observe CPU, memory, and process activity. This polling-based approach introduces sampling latency, misses short-lived events such as rapid process spawning, and adds unnecessary overhead. This project proposes Kernel Monitor, a lightweight Linux Loadable Kernel Module (LKM) that observes system activity directly from within the kernel by hooking process lifecycle events (fork, exec, exit) using kernel tracepoints/kprobes and periodically sampling CPU and memory statistics of running tasks via the kernel's task_struct list. The collected data is buffered in kernel space using synchronized data structures and exposed to user space through a custom /proc entry implemented with the seq_file interface. A companion user-space C application reads this interface to display real-time process and resource statistics on a terminal dashboard, and raises alerts when configurable CPU, memory, or process-count thresholds are exceeded. The expected outcome is a working kernel module and monitoring utility, hosted on GitHub, that tracks system resource usage and process behavior in real time with minimal performance impact on Linux systems.

2. Problem Statement

(Clearly describe the problem that the project intends to address and its relevance to Operating Systems and Systems Programming.)

Existing system monitoring tools such as `top`, `ps`, and `vmstat` operate entirely in user space and gather information by repeatedly reading and parsing the `/proc` filesystem at fixed intervals. This design has key limitations: (1) sampling delay causes processes created and terminated between polling intervals to go undetected; (2) repeated parsing of textual `/proc` data adds unnecessary CPU and I/O overhead; and (3) user-space tools have no visibility into finer-grained kernel-level events such as scheduling decisions or context switches. As workloads become more dynamic, these limitations reduce the accuracy and usefulness of monitoring. This project addresses the problem by moving core monitoring logic into the Linux kernel itself, using kernel modules, tracepoints, and kernel data structures to capture process and resource events as they occur rather than after the fact.

3. Objectives

(List the specific objectives of the project.)

1. Design and implement a Loadable Kernel Module (LKM) that hooks into process creation and termination events using kernel tracepoints/kprobes.
2. Periodically collect per-process CPU time, memory usage, and state information by traversing the kernel's `task_struct` list.
3. Expose collected kernel-space statistics to user space through a custom `/proc` filesystem entry using the `seq_file` interface.
4. Build a user-space C application that reads the `/proc` interface, displays real-time system metrics, and raises alerts on threshold breaches.

4. Proposed Methodology

(Briefly explain how the project will be designed and implemented in Linux. Describe the major functionality, approach, and flow of the proposed solution.)

Kernel Monitor will be implemented as a Loadable Kernel Module (LKM) written in C and inserted into the running Linux kernel using `insmod`. On initialization, the module registers kprobes/tracepoints on process lifecycle functions to record process creation and termination events with timestamps. A kernel timer periodically walks the `task_struct` linked list (via `for_each_process`) to sample per-process CPU utilization and memory usage from `mm_struct`, storing results in a kernel-space buffer protected by a spinlock/mutex for safe concurrent access. This data is exposed to user space through a custom entry under `/proc/kernel_monitor` implemented with the `seq_file` API. A companion user-space C program, built with `ncurses`, polls this `/proc` entry and renders a live dashboard of running processes, CPU/memory usage, and recent fork/exit events, comparing sampled values against configurable thresholds to raise alerts. The system will be developed and tested on Ubuntu Linux using a standard kernel module build environment.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
Loadable Kernel Modules (LKM) — <code>insmod/rmmod</code>	Load and unload the Kernel Monitor module into the running kernel without rebooting
Kprobes / Tracepoints on fork and exit	Capture process creation and termination events as they occur
<code>task_struct</code> / <code>for_each_process()</code>	Traverse the kernel's process list to sample per-process CPU and memory usage
<code>/proc</code> filesystem (<code>seq_file</code> interface)	Expose kernel-collected statistics to user space through a

	custom /proc entry
Kernel Timers (timer_list)	Periodically trigger sampling of system resource usage inside the kernel
Spinlocks / Mutexes	Ensure safe concurrent access to shared kernel-space monitoring data

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Host operating system for developing, building, and testing the kernel module
C / C++	Implementation language for the kernel module and user-space monitoring application
Kernel Headers / Makefile	Compile and build the Loadable Kernel Module against the running kernel
GDB / dmesg / printk	Debug and log kernel module behavior during development
ncurses / GitHub	Terminal-based dashboard UI and version control/collaboration

8. Expected Outcome

(Describe the expected functionality, output, and outcome of the project.)

A functioning Kernel Monitor LKM that tracks process creation/termination and per-process CPU and memory usage in real time with minimal overhead compared to traditional polling-based tools. The project will deliver a working /proc-based kernel-to-user-space interface, a terminal dashboard displaying live system statistics, and a basic threshold-based alerting mechanism for detecting abnormal CPU/memory usage or excessive process creation. The complete source code, build instructions, and a demonstration will be hosted on GitHub.

9. GitHub Repository

GitHub Repository Link: _

https://github.com/NAMITHRAO189/TechNova_OSSP

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____